

Contents

1	IRTC 使用手册 (中文·零基础完整版)	1
1.1	这本手册怎么读	1
2	第 0 章 最最基础：在开始之前	1
2.1	0.1 我们要解决一个什么样的现实问题？	1
2.2	0.2 你需要准备什么？	1
2.3	0.3 几个你会反复听到的词 (先混个眼熟，第 9 章有详细解释)	1
3	第 1 章 先把概念讲透 (零统计基础)	2
3.1	1.1 测验、问卷、量表到底在测什么？	2
3.2	1.2 为什么「把分数加起来」不够好？	2
3.3	1.3 看不见的 θ ，怎么测？——用「概率」搭桥	2
3.4	1.4 题目的两个「性格」：难度与区分度	3
3.5	1.5 二分题 vs 多级题	3
3.6	1.6 单维 vs 多维	3
3.7	1.7 IRTC 能做什么和不能做什么	3
4	第 2 章 准备你的电脑 (零编程基础)	3
4.1	2.1 R 和 RStudio 是什么？为什么要两个？	4
4.2	2.2 第一步：安装 R	4
4.3	2.3 第二步：安装 RStudio	4
4.4	2.4 认识 RStudio 的界面 (四块区域)	4
4.5	2.5 第一次运行命令：把控制台当计算器	4
4.6	2.6 第三步：安装 IRTC 包	5
4.7	2.7 第四步：加载包	5
4.8	2.8 跑通第一个完整例子 (用软件自带数据)	5
5	第 3 章 把数据整理成正确格式 (最关键的一章)	5
5.1	3.1 数据必须长成什么样？——「人 $\times$ 题」矩阵	6
5.2	3.2 二分计分 (0/1)	6
5.3	3.3 多级计分 (Likert 等)——必须从 0 开始连续编号	6
5.4	3.4 缺失值怎么标？——用 NA	6
5.5	3.5 反向计分题怎么处理？	7
5.6	3.6 把数据从 Excel / CSV 导入 R	7
5.7	3.7 导入后必做的检查 (5 个动作)	7
5.8	3.8 常见数据错误清单 (对照自查)	7
5.9	3.9 多维分析必备：Q 矩阵怎么写？	8
5.10	3.10 背景变量、分组、权重怎么单独存？	8
6	第 4 章 全量函数使用说明 (每个参数讲透)	8
6.1	4.1 irtc.mml(): 固定斜率模型 (Rasch / PCM / RSM)	8
6.2	4.2 irtc.mml.2pl(): 含区分度模型 (2PL / GPCM)	9
6.3	4.3 method: 三个引擎到底怎么选？	10
6.4	4.4 control: 控制参数全清单 (逐项解释)	10
6.5	4.5 查看结果的 4 个方法 (S3 方法)	11
7	第 5 章 读懂结果 (每个输出讲透)	12
7.1	5.1 summary(m) 大致分几块？	12
7.2	5.2 题目参数表 m\$xi 怎么读？	12
7.3	5.3 区分度怎么看？(仅 2PL/GPCM)	12
7.4	5.4 个人能力分 m\$person 怎么用？	13
7.5	5.5 潜在分布 m\$variance / m\$mean 怎么读？	13
7.6	5.6 模型比较：对数似然、AIC、BIC	13
7.7	5.7 加速后必看：m\$accuracy_report	13
7.8	5.8 路由信息：m\$routing	14
7.9	5.9 信度与测量精度：我的问卷「准」吗？	14
7.10	5.10 怎样判断「好题 / 坏题」并决定删改？	14
7.11	5.11 把 EAP 能力分接回你的原始数据	15

8	第 6 章 完整实战案例 (从文件到结论)	15
8.1	案例 A : 一份 0/1 的能力测验 (最常见)	15
8.2	案例 B : 一份 5 点 Likert 满意度问卷	16
8.3	案例 C : 多维 (两个分量表)	16
8.4	案例 D : 大数据 + 加速 + 核查	16
8.5	案例 E : 分组比较 (如男女差异)	17
8.6	案例 F : 从一份 Excel 到一页报告 (完整叙事 , 强烈建议照做一遍)	17
9	第 7 章 数学定义与原理 (为没学过统计的人解释)	18
9.1	7.1 概率 : 先统一一个最基本的词	18
9.2	7.2 逻辑斯蒂曲线 : IRT 的「S 形」核心	18
9.3	7.3 似然 : 「哪套参数最像我看到的数据 ? 」	19
9.4	7.4 把能力「积分掉」: 边际似然 (MML)	19
9.5	7.5 EM 算法 : 先猜能力、再修题目 , 来回迭代	19
9.6	7.6 多维与「维度灾难」	19
9.7	7.7 流式引擎 : 为什么能算百万人	19
9.8	7.8 受控精度 : 用「预算」换速度 , 用「核查」保安全	19
9.9	7.9 模型识别 : 为什么要「固定一些东西」	20
9.10	7.10 信息准则 : 怎么公平比较模型	20
10	第 8 章 全量报错解决手册	20
10.1	8.1 数据与参数类	20
10.2	8.2 引擎与路由类	20
10.3	8.3 数值与收敛类	21
10.4	8.4 安装与编译类	21
11	第 9 章 术语表 (中英对照)	21
12	第 10 章 推荐参考文献与书目	23
13	第 11 章 IRTC 的技术路线、理论基础与优化效果	23
13.1	11.1 总体技术路线	23
13.2	11.2 精确性与稳定性原则	24
13.3	11.3 已达到的效果	24
14	第 12 章 常见问题解答 (FAQ)	24
15	第 13 章 把结果写进报告的模板	25
15.1	14.1 方法部分 (Methods) 模板	25
15.2	14.2 题目质量表 (建议放附表)	25
15.3	14.3 信度与精度	25
15.4	14.4 人群比较 / 潜在回归	25
15.5	14.5 加速与可信度 (若用了 fast)	25
15.6	14.6 一句话写「为什么用 IRT 而不是总分」	25
16	附录	26
16.1	附录 A : 一页速查卡	26
16.2	附录 B : CSV 数据模板 (二分题)	26
16.3	附录 C : 新手常用 R 基础操作	26

1 IRTC 使用手册 (中文·零基础完整版)

一本写给「没学过编程、也没学过统计」的调查工作人员的项目反应理论 (IRT) 分析手册

这本手册假设你 : - 没写过任何代码 , 没用过 R ; - 没系统学过统计 , 听到「似然」「方差」会发怵 ; - 但你手里有一份问卷 / 测验数据 (比如 Excel 表) , 想知道「每道题好不好」「每个人水平多高」「不同人群有没有差异」。

这本手册保证 : 从「电脑上什么都没装」开始 , 一步一步带你装好软件、把数据弄成正确格式、跑出结果、并读懂每一个数字的含义。遇到报错 , 你能在本手册里查到怎么修。所有专业名词第

一次出现都会用大白话解释，并在最后的术语表里再解释一遍。

1.1 这本手册怎么读

- 完全新手：从第 0 章一直读到第 6 章，照着做一遍，你就能独立完成一次分析。第 7 章（原理）可以略读，想深入再回来。
- 会用电脑、只是不懂 IRT：重点读第 1 章（概念）、第 5 章（读结果）、第 6 章（案例）。
- 遇到报错：直接跳到第 8 章（报错手册）按信息查。
- 忘了某个词：查第 9 章（术语表）。

每一章末尾有「本章你应该掌握的」小结，帮你确认有没有读懂。

2 第 0 章 最最基础：在开始之前

2.1 0.1 我们要解决一个什么样的现实问题？

设想你是一名调查工作人员。你做了一份问卷或一场考试，收回了很多份答卷。现在你面前是一张大表格：每一行是一个人，每一列是一道题，格子里是这个人在这道题上的作答（比如对/错，或者「非常不同意 = 1 ... 非常同意 = 5」）。

你想回答这样一些问题：

1. 这些题目本身好不好？哪道题太难、哪道题太简单？哪道题能很好地区分高水平和低水平的人，哪道题谁来做都差不多（没有区分力）？
2. 每个被调查者的「真实水平」是多少？不是简单地数对了几道题，而是一个更科学的、考虑了「题目难易不同」的能力分数。
3. 不同人群有没有差异？比如男生女生、城市农村、实验组对照组，他们的平均水平、分布是否不同？
4. 能不能用一些背景信息（年龄、教育程度等）来解释能力？

IRT 这个软件包，就是用来回答上面这些问题的工具。它背后用的方法叫项目反应理论（Item Response Theory，简称 IRT）。

2.2 0.2 你需要准备什么？

- 一台电脑（Windows 或 Mac 都行）。
- 你的数据（最好是 Excel 或 CSV 文件，第 3 章会教你怎么整理）。
- 大约 1~2 小时，跟着本手册走一遍。
- 不需要任何编程或统计基础——本手册会从零教起。

2.3 0.3 几个你会反复听到的词（先混个眼熟，第 9 章有详细解释）

词	一句话理解
R	一个免费的统计软件（就像一个超强的计算器 + Excel）。
R 包（package）	给 R 增加新功能的「插件」。IRT 就是一个包。
函数（function）	一条「做某件事」的命令，比如「估计模型」。你给它原料（数据），它给你结果。
参数（argument）	给函数的「设置选项」，比如「用哪种模型」「数据是哪个」。
作答矩阵 / 响应矩阵（response matrix）	就是那张「人 × 题」的大表格。
能力 / 潜在特质（ θ , theta）	我们想测但看不见的东西，比如「数学水平」「满意度」。
题目参数	描述一道题「性格」的数字，主要是难度和区分度。

本章你应该掌握的：你大概知道了 IRT 是干嘛的（分析问卷/测验数据），以及你需要准备一台电脑和一份数据。下面我们正式开始。

3 第 1 章 先把概念讲透 (零统计基础)

这一章不碰电脑，只讲道理。读懂了，后面操作才不会「知其然不知其所以然」。

3.1 1.1 测验、问卷、量表到底在测什么？

当你做一份数学测验，你真正关心的不是「这个人这 20 道题对了几道」，而是这个人的数学水平到底有多高。「数学水平」是一个看不见、摸不着的东西——我们叫它潜在特质或潜在能力，在公式里通常用希腊字母 θ (读作「西塔」, theta) 表示。

同理：- 一份抑郁量表测的是看不见的「抑郁程度」；- 一份满意度问卷测的是看不见的「满意度」；- 一场英语考试测的是看不见的「英语水平」。

题目 (item) 只是一个「窗口」：我们通过一个人在很多道题上的表现，去间接推断那个看不见的 θ 。这就是 IRT 的根本思路。

3.2 1.2 为什么「把分数加起来」不够好？

最朴素的做法是总分：数对了几道题，或者把 Likert 各题 (1~5 分) 加起来。这叫经典测量理论 (CTT)。它简单，但有几个大问题：

问题一：每道题难度不一样，但总分一视同仁。假设甲做对了 5 道很难的题，乙做对了 5 道很简单的题。总分都是 5，但显然甲的水平更高。总分法看不出这个差别，IRT 能。

问题二：分数依赖于「这套题」，换一套题就不可比。用难卷子考出来的 60 分，和用容易卷子考出来的 60 分，不是一回事。IRT 把题目的难度和人的能力放在同一把尺子上，使得「人」的估计不依赖于具体用了哪套题 (这叫「参数不变性」，是 IRT 最重要的优点之一)。

问题三：总分无法告诉你「这道题好不好」。IRT 会给每道题估计出「难度」「区分度」，让你能挑出坏题、改进问卷。

一句话：CTT 关心「总分」，IRT 关心「题目和人各自的性质，并把它们放在同一把尺子上」。这就是为什么大型测评 (高考类、PISA、国际比较、心理量表标准化) 几乎都用 IRT。

3.3 1.3 看不见的 θ ，怎么测？——用「概率」搭桥

关键的桥梁是一句话：

一个人在一道题上答对 (或选高分) 的概率，取决于「他的能力 θ 」和「这道题的性质」。

直觉上很自然：- 能力 θ 越高 → 答对的概率越大；- 题目越难 → 答对的概率越小。

IRT 把这句直觉写成了精确的数学公式 (第 7 章会展开，这里只讲意思)。有了公式，我们就能反过来做一件神奇的事：

我们看得见每个人在每道题上的作答 (数据)，于是可以反推出那些看不见的东西——每个人的 θ 、每道题的难度和区分度。这个「反推」的过程叫估计 (estimation)。

3.4 1.4 题目的两个「性格」：难度与区分度

每道题最重要的两个性质：

① 难度 (difficulty，记作 b) - 通俗说：这道题有多难答对 (或多难选高分)。- 难度高的题，只有高能力的人才答得对；难度低的题，大部分人都答得对。- 在 IRT 里，难度和能力用同一把尺子衡量：如果一个人的能力 θ 正好等于某题的难度 b ，那他在这道题上答对的概率是 50%。- 数值通常在 -3 到 +3 之间：负数 = 容易，正数 = 难，0 = 中等。

② 区分度 (discrimination，记作 a) - 通俗说：这道题「分辨能力高低」的本事有多强。- 区分度高的题：高能力的人几乎都对，低能力的人几乎都错——一道题就能把人分开，是「好题」。- 区分度低 (接近 0) 的题：不管能力高低，答对概率都差不多——这道题「没用」，谁来都一样，应该改掉。- 数值通常在 0.5 到 2.5 之间：越大，区分力越强。注意：只有 2PL/GPCM 这类模型才估计区分度；Rasch/1PL 假设所有题区分度都一样 (都等于 1)。

生活类比：把「能力 θ 」想成「跳高水平」，把「题目难度 b 」想成「横杆高度」。横杆越高（题越难），跳过去越难。区分度 a 则像「横杆判定的灵敏度」：高区分度的横杆，差一点就判失败、够了就判成功，泾渭分明；低区分度的横杆，跳没跳过都模模糊糊。

3.5 1.5 二分题 vs 多级题

- 二分题 (dichotomous)：只有两种结果，对/错、是/否，编码成 0 / 1。例：选择题判对错。
- 多级题 / 多分题 (polytomous)：有多个有序等级。例：Likert 量表「非常不同意 / 不同意 / 一般 / 同意 / 非常同意」，编码成 0,1,2,3,4。
- 处理多级题需要更复杂的模型：PCM (偏序模型)、GPCM (广义偏序模型)。它们会给每道题估计多个阈值 (从一级跨到下一级的难度)。

3.6 1.6 单维 vs 多维

- 单维 (unidimensional)：整份问卷只测一个潜在特质。例：一份纯数学测验只测「数学能力」。
- 多维 (multidimensional)：一份问卷同时测多个特质。例：一份综合测验里，前 20 题测「语文」、后 20 题测「数学」；或一份心理量表有「焦虑」「抑郁」两个分量表。
- 在多维情形，你需要告诉软件「每道题属于哪个维度」——这个对应关系写在一个叫 Q 矩阵的表里 (第 3.9 节详解)。
- 本手册和 IRTC 主要处理简单结构 (题间多维)：每道题只属于一个维度 (不跨维)。这是最常见的情形。

3.7 1.7 IRTC 能做和不能做什么

能做：- 单维 / 多维的 Rasch(1PL)、PCM、RSM、2PL、GPCM 模型估计；- 估计每道题的难度 (和区分度)；- 估计每个人的能力 (EAP 分数)；- 潜在回归：用背景变量 (年龄、性别...) 解释能力；- 多组：比较不同人群 (每组各自的能力分布)；- 个案权重：抽样调查里给每个人不同权重；- 大数据：内置流式引擎，普通笔记本也能算百万人。

暂不直接做 (需要别的软件)：- 3PL (带猜测参数) 模型；- 题内多维 (一道题同时载多个维度的复杂结构，需用 `method="grid"` 且功能受限)；- 认知诊断、DIF 自动检验等高级专题 (可结合其他包)。

本章你应该掌握的：① θ 是看不见的能力；② IRT 通过「作答概率」把「人的能力」和「题的难度/区分度」放在同一把尺子上反推；③ 难度 = 多难答对，区分度 = 分辨高低的本事；④ 二分题编码 0/1，多级题编码 0,1,2,...；⑤ 多维要用 Q 矩阵指明每题属于哪个维度。

4 第 2 章 准备你的电脑 (零编程基础)

这一章手把手带你装好软件、并第一次「跑通」一段命令。不需要任何基础。

4.1 2.1 R 和 RStudio 是什么？为什么要两个？

- R：真正干活的统计软件 (免费、开源)。你可以把它想成一个「极其强大的计算器 + Excel + 统计引擎」。但 R 自带的界面很「素」，对新手不友好。
- RStudio：一个给 R 用的、好看好用的工作台 (界面)。你实际操作的是 RStudio，它在背后调用 R。
- 类比：R 是发动机，RStudio 是带方向盘、仪表盘的整车。你开车，但发动机在底下干活。两个都要装，先装 R 再装 RStudio。

4.2 2.2 第一步：安装 R

1. 打开浏览器，访问 <https://cran.r-project.org> (这是 R 的官方网站，叫 CRAN)。
2. 在最上方选择你的系统：
 - Windows：点 Download R for Windows → base → Download R-x.x.x for Windows，下载后双击安装，一路「下一步」即可。
 - Mac：点 Download R for macOS → 下载对应芯片的 .pkg (Apple 芯片选 arm64，旧 Intel 选 Intel)，双击安装。
3. 装完后你的电脑里就有了 R。但先别打开 R 自带的界面，我们用 RStudio。

4.3 2.3 第二步：安装 RStudio

1. 访问 <https://posit.co/download/rstudio-desktop/>。
2. 下载 RStudio Desktop (免费版), 双击安装, 一路默认。
3. 装完后, 打开 RStudio (不是 R)。它会自动找到你刚装的 R。

4.4 2.4 认识 RStudio 的界面 (四块区域)

第一次打开 RStudio, 你会看到屏幕被分成几块。对新手最重要的是左下角的「控制台 (Console)」:

脚本编辑器 (你写命令的地方)	环境 / 历史 (看有哪些数据/变量)
控制台 Console (命令在这里运行、 结果在这里显示)	文件/图形/包/帮助 (装包、看图、看帮助)

- 控制台 (Console): 你输入一行命令、按回车, 它立刻执行并显示结果。最简单的用法就在这里。
- 脚本编辑器 (Script): 当你要写很多行、想保存下来反复用时, 写在这里 (菜单 File → New File → R Script 新建)。在脚本里把光标放到某一行, 按 Ctrl+Enter (Mac 是 Cmd+Enter) 就能把那行送到控制台运行。
- 环境 (Environment): 显示你现在内存里有哪些数据、变量。
- 右下角: 装包、看图、看帮助文档的地方。

小贴士: 命令前面带 # 的是注释, R 会忽略它, 是给人看的说明。本手册代码里的中文注释都是这个作用。

4.5 2.5 第一次运行命令：把控制台当计算器

在控制台里输入下面这行, 按回车:

```
1 + 1
```

你会看到:

```
[1] 2
```

[1] 是「这是第 1 个结果」的意思, 2 是答案。恭喜, 你已经运行了第一条 R 命令。

再试一个——把一个数存起来 (叫「赋值」), 用 <- (小于号加减号, 像个箭头):

```
x <- 5      # 把 5 存进一个叫 x 的盒子
x * 2       # 取出 x 乘以 2
```

输出 [1] 10。这里 x 叫变量, <- 叫赋值 (把右边的东西装进左边的盒子)。这是 R 最核心的操作, 请务必理解。

4.6 2.6 第三步：安装 IRTC 包

IRTC 是一个「插件 (包)」, 要先装进 R。你拿到的是一个文件 IRTC_0.1.0.tar.gz (源码包)。装它需要电脑能「编译 C++」(因为 IRTC 有高性能的底层代码):

先装编译工具 (只需一次): - Windows: 访问 <https://cran.r-project.org/bin/windows/Rtools/>, 下载与你 R 版本匹配的 Rtools, 安装。- Mac: 打开「终端 (Terminal)」应用, 输入 xcode-select --install 回车, 按提示安装命令行工具。- Linux: 安装 r-base-dev (如 sudo apt-get install r-base-dev)。

再安装 IRTC (在 RStudio 控制台里):

```
# 把引号里换成你电脑上 IRTC_0.1.0.tar.gz 的真实路径
install.packages("C:/Users/你的名字/Downloads/IRTC_0.1.0.tar.gz",
                 repos = NULL, type = "source")
```

「路径」是什么？就是文件在电脑里的「地址」。比如下载到「下载」文件夹，Windows 路径常是 C:/Users/你的名字/Downloads/IRTC_0.1.0.tar.gz，Mac 常是 /Users/你的名字/Downloads/IRTC_0.1.0.tar.gz。注意 R 里路径用正斜杠 /，不要用反斜杠 \。

找路径的笨办法：在「文件资源管理器/访达」里找到那个文件，右键看属性/简介里的位置；或把文件拖进 RStudio 控制台，它会自动显示路径。

IRTC 还依赖另外两个包 (Rcpp、RcppArmadillo)，如果安装时提示缺少，先装它们：

```
install.packages(c("Rcpp", "RcppArmadillo"))
```

4.7 2.7 第四步：加载包

安装是「装到电脑里」(只需一次)；每次新开 RStudio 用 IRTC 前，都要先「加载」它 (告诉 R 这次要用它)：

```
library(IRTC)
```

没有报错就成功了。如果报「there is no package called 'IRTC」，说明上一步安装没成功，回到 2.6。

4.8 2.8 跑通第一个完整例子 (用软件自带数据)

IRTC 自带了一些练习数据，不用你准备，直接能跑。在控制台逐行运行：

```
library(IRTC)           # 加载包
data(data.sim.rasch)     # 调出自带数据：2000 人 × 40 题，0/1 作答
dim(data.sim.rasch)      # 看数据有多大 -> 应显示 2000 40
mod <- irtc.mml(resp = data.sim.rasch) # 估计 Rasch 模型，结果存进 mod
summary(mod)             # 打印结果摘要
```

如果屏幕滚出一堆迭代信息、最后打印出题目参数和方差，你已经完整跑通了一次 IRT 分析！第 5 章会教你读懂这些结果。

本章你应该掌握的：① 先装 R 再装 RStudio，实际用 RStudio；② 命令在「控制台」里运行；③ <- 是赋值 (装进盒子)，# 是注释；④ install.packages(...) 装包 (一次)，library(IRTC) 加载包 (每次)；⑤ 你已经能跑通自带数据的例子。

5 第 3 章 把数据整理成正确格式 (最关键的一章)

80% 的报错来自数据格式不对。这一章请务必逐节读、对照检查自己的数据。

5.1 3.1 数据必须长成什么样？——「人 × 题」矩阵

IRTC 要求的数据是一张长方形表格，规则极其明确：

- 每一行 = 一个被调查者 (一个人)；
- 每一列 = 一道题 (一个项目 item)；
- 每个格子 = 这个人在这道题上的作答 (已经编码成数字)。

举例，5 个人做 4 道二分题 (对 =1，错 =0)：

(人)	题 1	题 2	题 3	题 4
张三	1	1	0	0
李四	1	0	0	0
王五	1	1	1	1
赵六	0	1	0	1
孙七	1	1	1	0

在 R 里，这张表是一个 矩阵 (matrix) 或 数据框 (data.frame)。注意：表里只能放「作答」，不能混进姓名、学号、年龄这些列 (那些要单独存，见 3.10)。

5.2 3.2 二分计分 (0/1)

- 答对 / 是 / 选中 → 1
- 答错 / 否 / 未选 → 0

千万不要用「对/错」「Y/N」这种文字，必须是数字 0 和 1。

5.3 3.3 多级计分 (Likert 等) ——必须从 0 开始连续编号

这是新手最常踩的坑。多级题 (比如 5 点 Likert) 必须编码成从 0 开始的连续整数：

选项	正确编码	错误编码
非常不同意	0	1 [X]
不同意	1	2 [X]
一般	2	3 [X]
同意	3	4 [X]
非常同意	4	5 [X]

为什么必须从 0 开始？因为 GPCM/PCM 模型内部把类别当作「0,1,2,...」来处理。如果你用 1~5，软件会以为最低档是 1 (而 0 这一档「从没人选」)，导致结果错误或报错。

如果你的数据是 1~5，怎么改成 0~4？整列减 1：

```
# 假设 dat 是你的数据，所有题都是 1~5，统一减 1 变成 0~4
dat <- dat - 1
```

还要注意：每道题的最高档不必相同 (有的题 0-3、有的 0-4 都行)，但每道题内部必须是连续的——如果某道题理论上 有 0,1,2,3 四档，但实际数据里从没人选过 2，会造成「空类别」，可能出问题 (见第 8 章)。

5.4 3.4 缺失值怎么标？——用 NA

如果某个人某道题没作答 / 跳过 / 数据缺失，在 R 里要标成 **NA** (Not Available，大写，不加引号)。不要用空格、0、-9、999 之类代替缺失——那会被当成真实作答，污染结果。

如果你的原始数据用 -9 或 999 表示缺失，导入后要替换：

```
dat[dat == -9] <- NA      # 把所有 -9 替换成 NA
dat[dat == 999] <- NA
```

5.5 3.5 反向计分题怎么处理？

很多量表有反向题。例：满意度量表里，「我经常感到沮丧」这道题，选「非常同意」反而说明满意度低。这类题在分析前必须先反转，让所有题的「高分」都指向同一方向 (都代表高能力/高满意)。

5 点量表 (0~4) 反转公式：新值 = 4 - 旧值 (最高档减去原值)。

```
# 假设第 3、7 题是反向题，量表是 0~4 (最高 4)
dat[, c(3, 7)] <- 4 - dat[, c(3, 7)]
```

这一步必须在估计之前做。忘了反转，区分度会出现负数或结果混乱。

5.6 3.6 把数据从 Excel / CSV 导入 R

你的数据多半在 Excel 里。最稳妥的流程：

第一步：在 Excel 里整理好，另存为 CSV。- 删掉多余的标题行、说明行，只留一行表头 (题目名) + 数据；- 确保没有姓名/学号这些非作答列混在中间 (如果有，放到最左边或最右边，导入后再删，见 3.10)；- 把缺失格留空或填成统一的标记 (如 -9)；- Excel 菜单「文件 → 另存为 → CSV (逗号分隔)」，存成比如 mydata.csv。

第二步：在 R 里读进来。用 read.csv：

```
# header=TRUE 表示第一行是表头 (题目名)，不是数据
dat <- read.csv("C:/Users/你的名字/Documents/mydata.csv", header = TRUE)
```


读进来后，dat 就是你的数据。注意路径用 /，文件名要对。

更简单的办法（推荐新手）：在 RStudio 右上角的 Environment 面板点 Import Dataset → From Text (base/readr)，用图形界面选文件、预览、点 Import。它会自动生成 read.csv 命令。

第三步：把数据框转成矩阵（很多情况需要）：

```
dat <- as.matrix(dat) # 转成矩阵
```

5.7 3.7 导入后必做的检查（5 个动作）

数据一进来，先体检，再分析：

```
dim(dat)           # 看行数和列数 -> 行 = 人数，列 = 题数，对不对？
head(dat)          # 看前 6 行长什么样，有没有乱
str(dat)           # 看每列是不是数字（应是 num/int，不能是 chr 文字）
range(dat, na.rm=TRUE) # 看最小值和最大值 -> 二分题应是 0~1，5 点题应是 0~4
sum(is.na(dat))     # 看一共有多少缺失值
table(dat)         # 看每个数值各出现多少次 -> 有没有意外的值
```

逐项核对：- dim 的两个数对不对（人数、题数）？- range 是不是在你预期的范围（0~1 或 0~4）？出现 5、-9 等说明编码没改对（回 3.3/3.4）。- str 里每列是不是 num 或 int？如果是 chr（字符），说明混进了文字（比如「对/错」没转成数字），要先处理。

5.8 3.8 常见数据错误清单（对照自查）

现象	原因	解决
range 出现 5（5 点题）	用了 1~5 没减 1	dat <- dat - 1
range 出现 -9/999	缺失没转成 NA	dat[dat==-9] <- NA
str 显示某列是 chr	混进了文字	检查该列，转成数字或删除
行数远多于人数	Excel 里有空行	删掉空行重存
列数多了几列	混进了姓名/学号列	见 3.10，删除非作答列
区分度出现负数	反向题没反转	见 3.5 先反转
某道题结果异常	该题某类别零计数	见第 8 章

5.9 3.9 多维分析必备：Q 矩阵怎么写？

如果你的问卷测多个维度（比如「语文」和「数学」两个分量表），你要告诉软件每道题属于哪个维度。这通过一张 Q 矩阵实现：

- Q 矩阵的行 = 题，列 = 维度；
- 格子是 0 或 1：第 j 题属于第 d 维就在 Q[j,d] 放 1，否则放 0；
- 简单结构下，每行只有一个 1（每题只属于一个维度）。

例：8 道题，前 4 题测维度 1（语文），后 4 题测维度 2（数学）：

```
Q <- cbind(c(1,1,1,1,0,0,0,0), # 第 1 列：维度 1，前 4 题为 1
           c(0,0,0,0,1,1,1,1)) # 第 2 列：维度 2，后 4 题为 1
```

```
Q
#      [,1] [,2]
# [1,]    1    0    <- 题 1 属于维度 1
# ...
# [5,]    0    1    <- 题 5 属于维度 2
```

单维分析不需要 Q 矩阵（默认就是单维），或者写一个全 1 的单列 Q：

```
Q <- matrix(1, nrow = ncol(dat), ncol = 1) # 所有题都在维度 1
```

5.10 3.10 背景变量、分组、权重怎么单独存？

「年龄、性别、地区、抽样权重」这些不是作答，绝对不能放进作答矩阵。把它们单独存成向量或单独的列：

```
# 假设原始 raw 表里第 1 列是性别、第 2 列是地区、第 3 列起才是题目
gender <- raw[, 1] # 分组变量 (多组分析用)
region <- raw[, 2]
resp <- as.matrix(raw[, -(1:2)]) # 去掉前两列, 剩下的才是作答矩阵
```

- 分组变量 group：一个长度等于人数的向量，标明每个人属于哪一组（如性别）。
- 协变量 Y：背景变量做成矩阵，用于潜在回归（见 4.1 的 Y）。
- 权重 pweights：抽样调查的个案权重，一个长度等于人数的非负向量。

本章你应该掌握的：① 数据是「行 = 人、列 = 题」的纯作答矩阵；② 二分 0/1，多级从 0 连续编号；③ 缺失标 NA；④ 反向题先反转；⑤ 用 read.csv 导入、as.matrix 转换、导入后必做 5 项体检；⑥ 多维要写 Q 矩阵；⑦ 背景/分组/权重单独存，不混进作答矩阵。

6 第 4 章 全量函数使用说明（每个参数讲透）

IRTC 对外只有 2 个估计函数（irtc.mml、irtc.mml.2pl）和 4 个查看结果的方法（summary、print、logLik、anova）。本章把每个参数「是什么、放什么、举例」讲清楚。

6.1 4.1 irtc.mml()：固定斜率模型（Rasch / PCM / RSM）

什么时候用它？当你想用不估计区分度的模型——即假设所有题区分力相同。这类模型最经典、最稳健，包括：- 1PL / Rasch：二分题，只估难度；- PCM（偏序模型）：多级题，估每个阈值；- RSM（评定量表模型）：多级题，且各题阈值结构相同（适合同一套 Likert 选项的整份问卷）。

最简单的调用：

```
mod <- irtc.mml(resp = dat) # 默认 1PL/Rasch
mod <- irtc.mml(resp = dat, irtmodel = "PCM") # 多级题用 PCM
```

全部常用参数逐个解释：

参数	这是什么、放什么	例子
resp	你的作答矩阵（人 × 题）。必填。	resp = dat
irtmodel	选哪种模型："1PL"（二分 Rasch）、"PCM"（多级偏序）、"PCM2"（PCM 的另一种参数化）、"RSM"（评定量表）。	irtmodel = "PCM"
Q	Q 矩阵（题 × 维），指明每题属于哪维。单维可不填。	Q = Qmat
ndim	维度数。不写 Q 时用它说明有几维（默认 1）。	ndim = 2
Y	潜在回归的背景变量矩阵（人 × 变量）。想「用年龄/性别解释能力」时填。一般第一列放全 1（截距）。	Y = cbind(1, age)
group	分组向量（长度 = 人数），做多组比较时填，如性别。	group = gender
pweights	个案权重（长度 = 人数，非负），抽样调查填。	pweights = w
xsi.fixed	把某些题目参数固定成已知值（高级用法，做等值/锚题时用）。	一般留空
variance.fixed	固定（协）方差的某些元素（高级）。	一般留空
est.variance	是否估计潜在能力的方差（默认 TRUE）。	est.variance = TRUE
pid	每个人的 ID（不影响估计，只是给结果贴标签）。	pid = id_vec
verbose	是否在屏幕打印迭代进度（默认 TRUE）。嫌烦设 FALSE。	verbose = FALSE

参数	这是什么、放什么	例子
control	控制参数列表，微调算法（见 4.4）。	control = list(maxiter=500)

完整例子：

```
# 多维 PCM：用 Q 矩阵，关闭进度打印
```

```
m <- irtc.mml(resp = dat, irtmodel = "PCM", Q = Qmat, verbose = FALSE)
```

```
# 潜在回归：用「是否城市 (city=0/1)」预测能力
```

```
m <- irtc.mml(resp = dat, Y = cbind(1, city))
```

```
# 抽样权重
```

```
m <- irtc.mml(resp = dat, pweights = w)
```

返回什么？一个 irtc 类的「结果对象」，你把它存进 m，再用第 5 章的方法去读。里面有 m\$xi（题目参数）、m\$person（个人能力）、m\$variance（方差）、m\$ic（信息准则）等。

6.2 4.2 irtc.mml.2pl()：含区分度模型（2PL / GPCM）

什么时候用它？当你想估计每道题的区分度（不假设所有题一样），或处理大数据。包括：- 2PL：二分题，估难度和区分度；- GPCM（广义偏序）：多级题，估区分度和各阈值；- 变体：GPCM.design、2PL.groups、GPCM.groups（高级，见帮助页）。

这是大数据「流式引擎」的入口。

最简单的调用：

```
m <- irtc.mml.2pl(resp = dat, irtmodel = "2PL")           # 二分 2PL
m <- irtc.mml.2pl(resp = dat, irtmodel = "GPCM", Q = Qmat) # 多级 GPCM
```

在 4.1 参数之外，新增/重点参数：

参数	这是什么、放什么	例子
irtmodel	"2PL"、"GPCM"、 "GPCM.design"、 "2PL.groups"、 "GPCM.groups"。	irtmodel = "GPCM"
method	选哪个计算引擎："auto"（自动，推荐）、"grid"（标准全网格）、"streaming"（大数据流式）。详见 4.3。	method = "auto"
est.slopegroups	让某些题共享同一个区分度（把题分组）。	高级，一般留空
E	区分度的设计矩阵（GPCM.design 用）。	高级
control	控制参数，流式与加速选项都在这里（见 4.4）。	control = list(fast=TRUE)

完整例子：

```
# 自动选引擎（推荐）
```

```
m <- irtc.mml.2pl(resp = dat, irtmodel = "GPCM", Q = Qmat)
```

```
m$routing      # 看它选了哪个引擎、为什么
```

```
# 多组：比较男女，每组各自的能力分布
```

```
m <- irtc.mml.2pl(resp = dat, irtmodel = "2PL", Q = Qmat,
                  group = gender, method = "streaming")
```

```
m$variance     # 多组时是「列表」，每组一个协方差
```

```
m$gmean        # 各组的平均能力（参照组固定为 0）
```

6.3 4.3 method：三个引擎到底怎么选？

irtc.mml.2pl 有三种「计算引擎」，结果目标一致，只是速度和适用范围不同：

- **"grid"** (网格引擎)：最标准、最稳的全网格数值积分。任何受支持的模型都能用，小数据首选。缺点：维度高、数据大时会慢、占内存。
- **"streaming"** (流式引擎)：为大型、题间多维 (简单结构)、2PL/GPCM 设计的高性能引擎，内存占用很小 (百万人也不爆内存)，支持协变量/组/权重。遇到它不支持的模型 (如题内多维) 会报错提示你改用 grid。
- **"auto"** (自动，默认)：软件自己预测哪个引擎更快 (根据人数、题数、维度、节点数、协变量模式数)，然后自动选。新手就用默认的 auto，不用纠结。结果里 m\$routing 会告诉你它选了谁、为什么。

一句话：不确定就用 **method="auto"** (默认)。只有当你明确知道要强制某引擎时才手动指定。

6.4 4.4 control：控制参数全清单 (逐项解释)

control 是一个列表，用来微调算法。写法是 control = list(参数 1 = 值 1, 参数 2 = 值 2)。新手大多数情况留空即可，下面解释每一项，按「你最可能用到」排序。

A. 基本收敛/积分控制

项	含义	默认	什么时候改
maxiter	最大迭代次数 (算法最多算多少轮)	1000	提示「未收敛」时调大
conv	参数收敛阈值 (变化小于它就停)	1e-4	一般不动
convD	偏差收敛阈值	1e-3	一般不动
nodes	每个维度的积分节点 (一个数值向量)	seq(-6,6,len=21)	高维想更快可减少；想更精可增多
Msteps	每轮内部小迭代数	4	一般不动
acceleration	EM 加速： "none"/"squarem"/"Ramsay"/"Yu"	"none"	收敛慢时用 "squarem" 提速
ridge	协方差「岭」正则 (数值稳定用的少量)	0	协方差报奇异时设个小值如 1e-4
verbose/progress	是否打印进度	TRUE	嫌烦设 FALSE
n_threads	并行线程数	2 (最大 2)	可设为 1 以使用单线程；为符合 CRAN 政策不可超过 2

B. 大数据加速 (流式 fast 模式)

项	含义	默认
fast	是否开启近似加速 (剪掉不重要的积分节点)	FALSE (精确)
mass_budget	加速的「预算 ϵ 」：愿意丢掉多少先验质量换速度。注意：它是预算，不是保证误差 (真正的误差由下面的核查报告给出)	1e-3
verify	加速后的误差核查方式： "stratified" (分层抽样，推荐)/"random"/FALSE (不查)	"stratified"
verify_n	核查用多少人	3000
verify_seed	核查抽样的随机种子 (保证每次核查可复现)	1
refine	如果核查没达标，自动收紧预算重算直到达标	FALSE
tol_deviance / tol_eap / tol_moment / tol_par	核查时各指标的容差 (偏差/能力分/矩/参数)	各有默认

C. 多组与正则 (高级)

项	含义	默认
group_structure	多组协方差结构 : "full" (每组独立协方差) / "mean" (共享协方差、只放开组均值)	"full"
reg	统计正则 (流式): 列表, 可含 slope_penalty (区分度惩罚)、slope_max (区分度上限)、sigma_shrink/sigma_shrink_pooled (协方差收缩)	关

加速 + 核查的标准写法 (大数据放心用):

```
m <- irtc.mml.2pl(resp = bigdat, irtmodel = "GPCM", Q = Qmat,
  control = list(fast = TRUE, mass_budget = 1e-3,
    verify = "stratified", verify_n = 3000, refine = TRUE))
m$accuracy_report$met # TRUE = 加速带来的实测误差在容差内, 结果可放心用
m$accuracy_report     # 看详细的误差报告 ( 中位数/95% 分位/最大值 )
```

6.5 4.5 查看结果的 4 个方法 (S3 方法)

把结果对象 (比如 m) 传给这些函数:

方法	作用	怎么用
summary(m)	打印可读的结果摘要 (题目参数、方差、信息准则)。最常用。	summary(m)
print(m) 或直接 m	简要打印对象。	m
logLik(m)	取对数似然 (衡量模型拟合好坏的数), 可配合 AIC(m)、BIC(m)。	AIC(m)
anova(m1, m2)	比较两个嵌套模型 (似然比检验), 看复杂模型是否显著更好。	anova(m1, m2)

本章你应该掌握的: ① irtc.mml 用于不估区分度的模型, irtc.mml.2pl 用于估区分度 (和大数据); ② resp 必填, irtmodel 选模型, Q 指明维度, Y/group/pweights 接背景/分组/权重; ③ method 默认 "auto" 即可; ④ control 里 fast+verify 是大数据加速且可信的关键; ⑤ 用 summary 等四个方法读结果。

7 第 5 章 读懂结果 (每个输出讲透)

跑完模型, summary(m) 会打印一堆数字。这一章教你逐块读懂, 并知道「什么样的数算好、什么样的算坏」。

7.1 5.1 summary(m) 大致分几块?

典型的 summary 输出包含:

1. 基本信息: 样本量、题数、维度、迭代次数、是否收敛。
2. 信息准则: Deviance、对数似然、AIC、BIC 等 (用于比较模型)。
3. 题目参数: 每道题的难度 (2PL/GPCM 还有区分度)。
4. 潜在分布: 能力的均值、方差/协方差。

逐块看下面各节。

7.2 5.2 题目参数表 m\$ xsi 怎么读？

直接看 m\$ xsi :

```
m$ xsi
#           xsi           se.xsi
# I1 -1.959017 0.06465854
# I2 -1.857027 0.06311470
# ...
```

- 行名 **I1**, **I2**, ... : 第 1 题、第 2 题.....
- **xsi** 列：这道题的难度参数 (item intercept/difficulty)。
 - 在本软件参数化里，模型是 $\text{logit}(\text{答对}) = \theta - \text{xsi}$ ，所以 **xsi** 就是难度：
 - * **xsi** 越大 → 题越难 (要更高能力才答对)；
 - * **xsi** 越小 (越负) → 题越容易。
 - 上例 I1 的 $\text{xsi} = -1.96$ ，说明这是一道很容易的题。
 - 经验范围：大约 -3 到 +3。超出这个范围 (如 ± 5) 通常说明题极端难/极端易，或数据有问题。
- **se.xsi** 列：这个难度估计的标准误 (standard error)，衡量「这个估计有多准」。越小越准。如果某题 **se** 特别大 (如 > 0.5)，说明这道题信息量少、估计不稳 (可能作答人太少或题太极端)。

多级题 (PCM/GPCM)：每道题会有多行/多个阈值参数 (从第 0 档跨到第 1 档、第 1 档跨到第 2 档...的难度)，命名类似 I1_Cat1、I1_Cat2。阈值应当随档次递增 (越高档越难够到)；若出现「阈值乱序」，是「无序阈值」现象，提示该题或某类别有问题。

7.3 5.3 区分度怎么看？(仅 2PL/GPCM)

irtc.mml.2pl 的结果里，区分度存在斜率数组 m\$B 中，或在 summary 的题目参数部分一并打印。读法：

- 区分度 **a** 越大，这道题分辨高低能力的本事越强。
- 经验范围：大约 0.5 ~ 2.5。
 - $a < 0.3$ ：区分力很弱，几乎是「坏题」，考虑删除或修订。
 - a 在 0.8 ~ 2：通常是「好题」。
 - a 出现负数：警报！几乎一定是反向题没反转 (回 3.5)，或该题与其余题测的方向相反。
 - a 极大 (如 > 4)：可能是「数据分离」 (这道题几乎完美区分)，估计不稳，可用 `reg=list(slope_max=4)` 设上限。

7.4 5.4 个人能力分 m\$ person 怎么用？

看 m\$ person (每行一个人)：

```
names(m$ person)
# "pid" "case" "pweight" "score" "max" "EAP" "SD.EAP"
```

- **pid / case**：人的编号。
- **score**：原始总分 (答对题数 / Likert 加总)，仅供参考。
- **max**：该人最高可能分。
- **EAP**：这个人的能力估计 (EAP = Expected A Posteriori，后验期望)。这就是 IRT 给出的、考虑了题目难易的「科学能力分」，可以替代原始总分用于排名、分组、相关分析。
 - EAP 的量纲：默认潜在能力均值约为 0、方差由模型估计。所以 EAP 正数 = 高于平均，负数 = 低于平均，类似标准分。
- **SD.EAP**：这个人能力估计的标准误 (不确定性)。越小越准。题目越多、信息越足，SD.EAP 越小。

取出来用：

```
ability <- m$ person$ EAP           # 拿到每个人的能力分向量
hist(ability)                       # 画能力分布直方图
# 也可以把 EAP 接回你的原始数据，做后续分析
```

7.5 5.5 潜在分布 m\$ variance / m\$ mean 怎么读？

- 单维：m\$ variance 是一个数，是能力的方差 (离散程度)。例 1.40 表示能力分布的方差为 1.40。
- 多维：m\$ variance 是一个协方差矩阵：对角线是各维方差，非对角线是维间协方差；换算成相关系数能看「两个维度有多相关」：


```
cov2cor(m$variance)      # 把协方差矩阵转成相关矩阵，看维度间相关
```

- 多组 (用了 group): `m$variance` 是一个列表，每组一个协方差矩阵；`m$gmean` 是各组的平均能力 (参照组固定为 0，其余组相对它)。这就是你做「人群比较」时要看的关键结果：哪个组平均能力更高、离散更大。

7.6 5.6 模型比较：对数似然、AIC、BIC

这些数 (在 `m$ic` 里，`summary` 也会打印) 用来比较不同模型谁更好：

- Deviance (偏差) = $-2 \times$ 对数似然：越小，拟合越好。但更复杂的模型总会拟合得更好，所以不能只看它。
- AIC、BIC：在「拟合好」和「别太复杂」之间权衡，越小越好。
 - 比较两个模型 (比如 1PL vs 2PL) 时，AIC/BIC 更小的那个更受青睐。
 - BIC 比 AIC 更「惩罚复杂度」，倾向选更简单的模型；样本大时常看 BIC。
- 嵌套模型 (一个是另一个的特例，如 Rasch 是 2PL 的特例) 可用似然比检验：

```
m1 <- irtc.mml(resp = dat)                # Rasch (简单)
m2 <- irtc.mml.2pl(resp = dat, irtmodel = "2PL") # 2PL (复杂)
anova(m1, m2)                             # 看 p 值：显著 (<0.05) 则复杂模型 2PL 明显更好
AIC(m1); AIC(m2)                          # 直接比 AIC
BIC(m1); BIC(m2)
```

7.7 5.7 加速后必看：m\$accuracy_report

如果你开了 `control=list(fast=TRUE)`，结果里会多一个 `m$accuracy_report` (误差核查报告)。这是判断「加速结果能不能信」的依据：

```
m$accuracy_report$met          # 总判定：TRUE = 实测误差在容差内，可放心用
m$accuracy_report$eap_abs      # 能力分误差的 中位数/95% 分位/最大值
m$accuracy_report$deviance_rel # 偏差的相对误差
m$accuracy_report$nodes_kept   # 加速后保留了多少积分节点
m$accuracy_report$nodes_full   # 原本有多少节点 (对比看省了多少)
```

- `met == TRUE`：加速带来的误差经过分层抽样实测，确认在你设定的容差内——结果可信。
- `met == FALSE`：误差超标。处理：调小 `mass_budget` (如 $1e-4$)，或设 `refine=TRUE` 让它自动收紧重算。
- 如果看到提示「`kept > 70% of nodes`」，说明加速没省多少，不如直接关掉 `fast` 用精确模式。

7.8 5.8 路由信息：m\$routing

用 `method="auto"` (或 `streaming`) 时，`m$routing` 告诉你软件选了哪个引擎、为什么：

```
m$routing$engine  # "grid" 或 "streaming"
m$routing$reason  # 文字理由，如 "streaming faster (pred 19.1x)"
```

这只是「幕后信息」，不影响结果正确性，了解即可。

7.9 5.9 信度与测量精度：我的问卷「准」吗？

调查工作者最常被问：「你这套题靠谱吗？」在 IRT 里，「靠谱」有两层含义。

第一层：个人能力估计有多准——看 **SD.EAP**。每个人的 **SD.EAP** (能力估计的标准误) 越小，这个人的分数越可信。一个粗略的经验：

- $SD.EAP < 0.3$ ：很准 (题目对这个能力水平提供了充足信息)；
- $0.3 \sim 0.5$ ：尚可；
- > 0.5 ：偏粗 (这个人所处的能力区间题目信息不足，可能因为题太少、或没有难度匹配他的题)。

```
mean(m$person$SD.EAP) # 平均测量精度，越小越好
hist(m$person$SD.EAP)  # 看哪些人估得不准 (往往是能力极高/极低者)
```


第二层：整份问卷的「边际信度」——类似传统的信度系数。可以用「能力的真实方差 ÷ (真实方差 + 平均测量误差方差)」估算一个边际信度 (marginal reliability), 含义类似 Cronbach's α , 越接近 1 越好 (一般 > 0.8 算可接受, > 0.9 算优秀):

```
# 单维近似算法:
var_theta <- as.numeric(m$variance)      # 能力真实方差
err_var   <- mean(m$person$SD.EAP^2)    # 平均测量误差方差
rel <- var_theta / (var_theta + err_var)  # 边际信度
rel
```

为什么 IRT 的信度比传统 α 更有用? 因为 IRT 能告诉你信度在不同能力水平上是不一样的: 题目大多是中等难度时, 对中等能力者测得准 (信度高), 对极端能力者测不准 (信度低)。这正是 SD.EAP 随能力变化所反映的。

7.10 5.10 怎样判断「好题 / 坏题」并决定删改?

把第 5.2、5.3 节的指标综合起来, 给每道题打分。一道好题通常满足:

指标	好题的范围	坏题的信号
区分度 a (2PL/GPCM)	0.8 ~ 2.0	< 0.3 (没区分力) 或 < 0 (方向反了)
难度 b / xsi	-2 ~ +2, 且与你的人群匹配	超出 ± 3 (太极端, 几乎全对或全错)
标准误 se.xsi	越小越好	很大 (如 > 0.5, 估计不稳)
多级阈值	随档次递增 (有序)	阈值乱序 (无序阈值)

实操：一次性筛出可疑题。

```
# 以 2PL 为例, 把区分度和难度并排看
a <- m$person                                     # ( 区分度在 summary 里; 此处示意流程 )
diff <- m$xsi$xsi                                # 各题难度
se <- m$xsi$se.xsi                               # 各题标准误
bad_extreme <- which(abs(diff) > 3)               # 太难/太易的题
bad_unstable <- which(se > 0.5)                  # 估计不稳的题
bad_extreme; bad_unstable                        # 这些题号需要人工复核
```

决策原则: - 区分度为负 → 几乎一定是反向题没反转, 先回 3.5 处理, 而不是删题。- 区分度极低 (<0.3) → 这道题没用, 考虑删除或重新编制。- 难度极端 → 如果你的人群里这道题几乎全对/全错, 它提供的信息少, 可删; 但如果你故意要测极端人群, 则保留。- 删题后要重新估计 (题目集变了, 参数会变)。

7.11 5.11 把 EAP 能力分接回你的原始数据

分析的最终目的常是「给每个人一个能力分, 再做后续研究」。把 EAP 接回去:

```
# 假设 raw 是含背景变量的原始数据框, 行序与作答矩阵一致
raw$ability <- m$person$EAP                      # 新增一列「能力分」
raw$ability_se <- m$person$SD.EAP                # 新增「能力分标准误」
```

```
# 之后就能像普通变量一样用, 例如看能力与年龄的相关:
cor(raw$ability, raw$age, use = "complete.obs")
# 或按性别比较能力均值:
tapply(raw$ability, raw$gender, mean)
```

本章你应该掌握的: ① m\$xsi 的 xsi 是难度 (大 = 难), se.xsi 是估计精度; ② 区分度大 = 好题, 负数 = 反向题没反转; ③ m\$person\$EAP 是每个人的科学能力分, SD.EAP 是其精度; ④ m\$variance/m\$gmean 看 (各组) 能力的离散和均值; ⑤ AIC/BIC 越小越好、anova 比较嵌套模型; ⑥ 加速后看 accuracy_report\$met 是否为 TRUE; ⑦ 用 SD.EAP 和边际信度判断问卷准不准; ⑧ 综合区分度/难度/标准误判断好题坏题; ⑨ 把 EAP 接回原始数据做后续分析。

8 第 6 章 完整实战案例 (从文件到结论)

每个案例都是「从一份数据文件，到一句能写进报告的结论」的完整流程。把你自己的数据套进去即可。

8.1 案例 A：一份 0/1 的能力测验 (最常见)

场景：你有 500 个学生做了 30 道选择题 (对 =1，错 =0)，存在 test.csv。你想知道：题目质量如何？每个学生水平多高？

```
library(IRT)
```

```
# 1) 读数据并体检
```

```
dat <- read.csv("test.csv", header = TRUE)
```

```
dat <- as.matrix(dat)
```

```
dim(dat); range(dat, na.rm = TRUE); table(dat) # 应是 500x30，值 0/1
```

```
# 2) 先跑简单的 Rasch
```

```
m1 <- irtc.mml(resp = dat)
```

```
# 3) 再跑 2PL ( 估区分度 )，并比较哪个更好
```

```
m2 <- irtc.mml.2pl(resp = dat, irtmodel = "2PL")
```

```
AIC(m1); AIC(m2) # AIC 更小的模型更优
```

```
anova(m1, m2) # 似然比检验
```

```
# 4) 读题目质量 ( 以 2PL 为例 )
```

```
m2$xi # 每题难度
```

```
# 区分度在 summary(m2) 里看；找出区分度 <0.3 的坏题、难度过偏的题
```

```
# 5) 拿每个人的能力分
```

```
ability <- m2$person$EAP
```

```
summary(ability) # 能力分布概况
```

能写进报告的结论示例：「2PL 模型的 AIC 显著低于 Rasch (似然比检验 $p < 0.001$)，说明各题区分度确有差异。第 7、19 题区分度低于 0.3，建议修订。学生能力 EAP 估计的范围为 ...」

8.2 案例 B：一份 5 点 Likert 满意度问卷

场景：300 人做了 10 道满意度题，每题 1~5 分，其中第 4、9 题是反向题，存在 survey.csv。

```
library(IRT)
```

```
dat <- as.matrix(read.csv("survey.csv", header = TRUE))
```

```
# 1) Likert 1~5 -> 必须改成 0~4
```

```
dat <- dat - 1
```

```
# 2) 反向题反转 ( 0~4，新值 =4-旧值 )
```

```
dat[, c(4, 9)] <- 4 - dat[, c(4, 9)]
```

```
# 3) 体检
```

```
range(dat, na.rm = TRUE) # 应为 0~4
```

```
# 4) 多级题用 GPCM ( 估区分度 ) 或 PCM ( 不估 )
```

```
m <- irtc.mml.2pl(resp = dat, irtmodel = "GPCM")
```

```
summary(m)
```

```
# 5) 检查区分度有没有负数 ( 验证反转是否正确 )
```

```
# 若有负数，说明还有反向题没处理，回头检查
```

结论示例：「满意度量表 GPCM 拟合良好，各题区分度在 0.7~1.8 之间，无负值 (反向题处理正确)。被访者满意度 EAP 的均值约为 0 (量表中点附近)，离散度 (方差) 为 ...」

8.3 案例 C：多维（两个分量表）

场景：一份问卷有 16 题，前 8 题测「焦虑」、后 8 题测「抑郁」，二分计分。

```
library(IRT)
dat <- as.matrix(read.csv("twoscale.csv", header = TRUE))

# 1) 写 Q 矩阵：前 8 题维 1(焦虑)，后 8 题维 2(抑郁)
Q <- cbind(c(rep(1,8), rep(0,8)),
           c(rep(0,8), rep(1,8)))
```

```
# 2) 估计多维 2PL
m <- irtc.mml.2pl(resp = dat, irtmodel = "2PL", Q = Q)
```

```
# 3) 看两个维度的相关
cov2cor(m$variance)      # 焦虑与抑郁的相关系数
```

```
# 4) 每个人的两个维度分数
m$person$EAP             # 多维时每人对应一行多维 EAP (见列名)
```

结论示例：「焦虑与抑郁两维相关为 0.62，说明中度相关但确为两个可区分的维度。」

8.4 案例 D：大数据 + 加速 + 核查

场景：5 万人、60 题、3 维 GPCM，精确模式太慢，想加速但要确保结果可信。

```
library(IRT)
# dat 已是 50000 x 60 的矩阵；Q 是 60x3 的 Q 矩阵
m <- irtc.mml.2pl(resp = dat, irtmodel = "GPCM", Q = Q,
                  method = "auto",
                  control = list(fast = TRUE, mass_budget = 1e-3,
                                verify = "stratified", verify_n = 3000, refine = TRUE))

m$routing              # 看用了流式引擎
m$accuracy_report$met   # 必须是 TRUE 才放心
m$accuracy_report$eap_abs # 看能力分的实测误差有多少
```

结论：若 met==TRUE，可写「采用受控精度流式估计，分层核查确认能力分的 95% 分位误差 < 0.01，远小于估计本身的不确定性，结果可靠」。

8.5 案例 E：分组比较（如男女差异）

场景：想比较男生女生在能力上的差异。

```
library(IRT)
# dat 是作答矩阵；gender 是长度 = 人数的向量（如 "M"/"F" 或 1/2）
m <- irtc.mml.2pl(resp = dat, irtmodel = "2PL", Q = Q,
                  group = gender, method = "streaming")

m$gmean              # 各组平均能力（参照组为 0，另一组相对它）
m$variance            # 各组协方差（列表）
```

结论示例：「以男生为参照组，女生的平均能力高 0.18 个标准单位；两组能力离散度相近。」

8.6 案例 F：从一份 Excel 到一页报告（完整叙事，强烈建议照做一遍）

这个案例把前面所有环节串成一个真实工作流。设想你刚收回一份纸质问卷的录入结果。

背景：一份「学习投入量表」，250 名学生，12 道题，4 点 Likert（原始编码 1= 从不、2= 偶尔、3= 经常、4= 总是），其中第 5、10 题是反向表述。原始录入在 engagement.xlsx，第 1 列是学号 id，第 2 列是性别 sex（男/女），第 3 列起是 12 道题。

第 0 步：在 Excel 里另存为 CSV。打开 engagement.xlsx，确认没有多余标题行，「文件 → 另存为 → CSV（逗号分隔）」存成 engagement.csv。

第 1 步：读入并分离背景变量与作答。

```
library(IRT)
raw <- read.csv("engagement.csv", header = TRUE) # 读进来 (含 id、sex 和 12 题)
dim(raw) # 期望 250 x 14 (2 背景 + 12 题)

id <- raw$id # 留下学号
sex <- raw$sex # 留下性别 (多组用)
resp <- as.matrix(raw[, 3:14]) # 第 3~14 列才是作答
```

第 2 步：编码与反转 (最容易错的一步)。

```
resp <- resp - 1 # 4 点 1~4 -> 0~3
resp[, c(5, 10)] <- 3 - resp[, c(5, 10)] # 反向题反转 (0~3, 新 = 3-旧)
range(resp, na.rm = TRUE) # 必须是 0~3
table(resp) # 看各档分布、有没有怪值
sum(is.na(resp)) # 看缺失多少
```

第 3 步：选模型并估计。多级题、想看区分度 → GPCM。

```
m <- irtc.mml.2pl(resp = resp, irtmodel = "GPCM", verbose = FALSE)
```

第 4 步：看题目质量。

```
m$ksi # 各题阈值 (难度), 看有没有极端值
# 在 summary(m) 里看区分度; 重点: 有没有负的区分度 (反转是否到位)
summary(m)
```

假设你发现所有区分度为正、在 0.6~1.7 之间、无极端难度——说明量表质量良好、反转正确。

第 5 步：评估信度与个人分。

```
var_theta <- as.numeric(m$variance)
rel <- var_theta / (var_theta + mean(m$person$SD.EAP^2))
rel # 边际信度, 比如 0.86 -> 可接受
engagement <- m$person$EAP # 每个学生的「学习投入」分
```

第 6 步：人群比较 (男女差异)。

```
mg <- irtc.mml.2pl(resp = resp, irtmodel = "GPCM",
  group = sex, method = "streaming", verbose = FALSE)
mg$gmean # 各组平均投入 (参照组为 0)
```

第 7 步：把分数接回、产出可汇报的表。

```
out <- data.frame(id = id, sex = sex,
  engagement = round(m$person$EAP, 2),
  se = round(m$person$SD.EAP, 2))
head(out)
write.csv(out, "engagement_scores.csv", row.names = FALSE) # 存成结果文件
```

第 8 步：写进报告 (示例段落)。> 「采用广义偏序模型 (GPCM) 对 12 道学习投入题进行项目反应理论分析。全部题目区分度为正且介于 0.6–1.7，难度分布合理，反向题处理正确。量表边际信度为 0.86，测量精度良好。以男生为参照，女生的平均学习投入高 0.21 个标准单位。各生的学习投入分 (EAP) 及其标准误差附表。」

这一整套——读入、分离、编码、反转、体检、估计、读题、评信度、比较、导出、汇报——就是你以后每次分析的固定动作。

本章你应该掌握的：把自己的数据替换案例里的 dat/resp/Q/gender，按固定动作 (读入 → 分离背景 → 编码 → 反转 → 体检 → 估计 → 读题目 → 评信度 → 人群比较 → 导出 → 汇报) 就能独立完成一次完整分析并产出报告。

9 第 7 章 数学定义与原理 (为没学过统计的人解释)

这一章把背后的数学用大白话讲一遍。看不懂公式没关系，先看每段开头的「人话解释」。读完你会明白软件「到底在算什么」，从而更会用、更会判断结果。

9.1 7.1 概率：先统一一个最基本的词

人话：概率就是「发生的可能性」，从 0 (绝不发生) 到 1 (一定发生)。0.5 就是一半一半。IRT 里反复出现的「答对的概率」，就是「这个人在这道题上答对的可能性有多大」。

9.2 7.2 逻辑斯蒂曲线：IRT 的「S 形」核心

人话：我们需要一个公式，能把「能力 θ 减去难度 b 」这个可正可负的数，变成一个 0 到 1 之间的「概率」。最常用的就是逻辑斯蒂函数，它画出来是一条平滑的 S 形曲线：

- 当能力远低于难度 ($\theta - b$ 很负) $\rightarrow$ 概率接近 0 (几乎答不对)；
- 当能力等于难度 ($\theta - b = 0$) $\rightarrow$ 概率正好 0.5；
- 当能力远高于难度 ($\theta - b$ 很正) $\rightarrow$ 概率接近 1 (几乎一定对)。

公式 (Rasch)：

$$P(\text{答对}) = 1 / (1 + e^{-(\theta - b)})$$

这里 e 是数学常数 (约 2.718)， e^x 是指数函数。你不需要会算，只要记住这条 S 曲线的形状和上面三句话。

2PL 在指数里多乘一个区分度 a ： $P = 1 / (1 + e^{-a(\theta - b)})$ 。 a 越大，S 曲线越「陡」——能力稍微变化，概率就剧烈变化，所以「区分力强」。

多级题 (GPCM) 用类似的指数形式，给每个相邻档次之间算一个跨越概率，再归一化成各档的概率 (细节见英文版/帮助页，原理一致)。

9.3 7.3 似然：「哪套参数最像我看到的数据？」

人话：假设我们先猜一组题目参数和能力。用这组猜测，我们能算出「在这组猜测下，恰好出现我手里这份真实数据的可能性」有多大——这个可能性就叫似然 (likelihood)。

- 如果某组猜测算出的似然很大，说明这组猜测「很能解释」我看到的 data——它「很像真的」。
- 估计的目标，就是找出让似然最大的那组参数——这叫极大似然估计。

打个比方：你看到地上一串脚印 (数据)，你猜「是谁走过的」(参数)。似然就是「如果真是张三走的，会留下这串脚印的可能性」。你会选那个最能解释脚印的人。

9.4 7.4 把能力「积分掉」：边际似然 (MML)

人话：每个人的能力 θ 我们其实不知道 (它正是要估的)。在估计题目参数时，我们不想被「不知道的 θ 」卡住，于是用一个数学技巧：把所有可能的 θ 按其出现的概率「加权平均」掉——这叫对 θ 积分，得到的结果叫边际似然。

- 我们假设全体人的能力服从一个正态分布 (钟形曲线，多数人能力中等、少数人很高或很低)。
- 「积分掉 θ 」=「考虑了每种能力值出现的可能性后，综合算出这道题数据的似然」。
- 这种方法叫 边际极大似然 (Marginal Maximum Likelihood, MML)，是 IRT 的核心算法。

9.5 7.5 EM 算法：先猜能力、再修题目，来回迭代

人话：MML 的最大化不能一步到位，于是用一个「来回迭代」的聪明办法，叫 EM 算法：

1. E 步 (期望)：在当前的题目参数下，算出「每个人最可能的能力分布」，并据此算出「每道题、每个档次大概被多少 (加权的) 人选了」——这叫期望计数。
2. M 步 (最大化)：把这些期望计数当成「已知」，去更新题目参数和能力分布的方差，让似然更大。
3. 回到第 1 步，用新参数再算……如此反复，直到参数几乎不再变化 (收敛)。

就像「先根据现有题目猜每个人的水平，再根据猜出的水平修正题目，再用修正后的题目重新猜水平……」直到稳定。IRTC 在这里用了很讲究的数值方法（解析梯度、牛顿法、必要时退回更稳的算法）保证又快又稳，还可选 SQUAREM 技巧加速收敛。

9.6 7.6 多维与「维度灾难」

人话：单维时， θ 是一个数，积分就是在一条线上取很多点求和。多维时（ D 个维度）， θ 是一个 D 维向量，积分要在一个 D 维的网格上求和——节点数是「每维点数」的 D 次方（ Q^D ）。维度一多，节点数指数爆炸：3 维 21 点就有 9261 个节点，4 维就有 19 万个。这叫维度灾难，是多维 IRT 慢的根源。

9.7 7.7 流式引擎：为什么能算百万人

人话：IRTC 的「流式引擎」用两招对付大数据：

1. 按维分解：当每道题只属于一个维度（简单结构）时，似然可以拆成各维分别算，先算便宜的「每维似然」，再在大网格上拼起来。
2. 逐人流式累加：不所有人的中间结果都存在内存里，而是算一个人、累加一个人、丢一个人，所以内存占用几乎和人数无关——这就是百万人也不爆内存的原因。

软件还会预测「网格引擎」和「流式引擎」哪个更快，自动选（`method="auto"`）。

9.8 7.8 受控精度：用「预算」换速度，用「核查」保安全

人话：即使有流式引擎，超大网格仍可能慢。于是提供近似加速：把那些「几乎没人会落在上面、对结果几乎没贡献」的网格节点剪掉，只保留重要的，从而提速。

- **mass_budget**（质量预算 ϵ ）：你允许剪掉多少「先验质量」。但它只是预算，不直接等于结果误差。
- 为了不误伤极端被试（他们的能力落在罕见区域，那里先验小但对他们很重要），剪枝同时考虑「样本里实际有人落在哪」（后验占用），并对每个小组分别保护，再合并。
- 真正的安全保证来自实测核查：算完后，软件抽一个有代表性的子样本（含普通人、小组、高权重者、极端答题者），分别用「完整网格」和「剪枝网格」算一遍，对比两者差多少，报告误差的中位数/95% 分位/最大值，并判定是否在你设的容差内（`met`）。这就把「近似」变成了「带实测误差报告的近似」，让你能放心。

9.9 7.9 模型识别：为什么要「固定一些东西」

人话：有些东西天生「分不开」。比如「把所有人能力 +1、同时把所有题难度 +1」，作答概率完全不变——数据无法区分这两种情形。为了让答案唯一，必须人为固定一个基准（叫「识别约定」）：

- 尺度：把能力的方差固定为 1（相当于规定「一个标准单位有多长」）。
- 位置：把某个基准（如参照组的均值）固定为 0（规定「零点在哪」）。
- 多组：题目参数跨组共享，作为把各组放到同一把尺子上的「锚」。

这些固定不影响你关心的相对结论（谁难谁易、谁高谁低），只是定下了「尺子的零点和单位」。

9.10 7.10 信息准则：怎么公平比较模型

人话：复杂模型（参数多）总能把数据拟合得更好，但可能「过拟合」（把噪声也学了）。AIC、BIC 在「拟合好」和「别太复杂」之间打分：分越低越好。BIC 对复杂度罚得更重，样本大时更稳。带权重时，用「有效样本量」（Kish 公式）代替人数。

本章你应该掌握的：① 逻辑斯蒂 S 曲线把「能力 - 难度」变成概率；② 似然 = 「这套参数有多像真数据」，估计 = 找最大似然的参数；③ MML 把未知能力积分掉；④ EM 来回迭代「猜能力 $\leftrightarrow$ 修题目」；⑤ 流式引擎靠「按维分解 + 逐人流式」省内存省时间；⑥ 加速用预算换速度、用实测核查保安全；⑦ 识别约定只是定下尺子的零点和单位。

10 第 8 章 全量报错解决手册

报错不可怕。每条都告诉你「你会看到什么 / 为什么 / 一步步怎么修」。先在本章用关键词搜你看到的英文信息。

10.1 8.1 数据与参数类

pweights must be non-negative, finite, length N, not all zero. - 含义：你给的个案权重 pweights 不合法。- 原因：权重长度不等于人数，或有负数 / NA，或全是 0。- 解决：检查 `length(pweights)` 是否等于行数；`any(pweights<0)`、`any(is.na(pweights))` 是否为真；`sum(pweights)` 是否 >0。修正后再跑。

Y contains missing values; resolve them before estimation. - 含义：你的协变量矩阵 Y 里有缺失值 NA。- 原因：背景变量（年龄、教育等）有空缺。软件不会替你猜，要求你先处理。- 解决：删除有缺失的个案，或用合理方法插补，使 Y 无 NA。

group contains missing values... / pweights contains missing values... - 同上：分组向量 / 权重里有 NA。补全或删除这些个案。

extreme case weights (max/mean > 50). (警告，不是错误) - 含义：权重极端不均，少数人的权重是平均的 50 倍以上。- 风险：极少数高权重个体可能主导整个结果。- 解决：核对权重是否正确；必要时对极端权重做「截断 (winsorize)」。结果仍会给出，但要谨慎解读。

collinear covariates (columns ...) are not identified. (警告) - 含义：协变量 Y 里有「共线」的列（比如两列完全一样，或一列能被其他列线性表示）。- 解决：删掉冗余的协变量列。被点名的列其系数会被置 0。

10.2 8.2 引擎与路由类

method='streaming' unsupported for this model (within-item / non-simple structure); use method='grid'. - 含义：你强制用了流式引擎，但你的模型它不支持（题内多维 / 非简单结构）。- 解决：改用 `method = "grid"`；或把 Q 矩阵改成简单结构（每题只属于一个维度）。

Multiple group estimation is not (yet) supported for ... (来自 grid 引擎) - 含义：网格引擎不支持多维 + 多组一起做。- 解决：改用流式引擎 `method = "streaming"`（它支持多维多组）；或把模型降为单维。

accuracy verification did not meet tolerance; see accuracy_report. (警告) - 含义：你开了加速 (`fast=TRUE`)，但实测核查发现误差超过了容差。- 解决：① 把 `mass_budget` 调小（如从 $1e-3$ 改 $1e-4$ ）；② 设 `refine = TRUE` 让它自动收紧重算；③ 看 `m$accuracy_report` 是哪个指标、哪个分位超标，针对性处理。

fast mode kept >70% of nodes; speed benefit is limited. (提示信息) - 含义：在你的数据上，剪枝省不了多少节点，加速收益很小却仍承担近似风险。- 解决：干脆关掉 `fast`（用默认的精确模式）。

10.3 8.3 数值与收敛类

现象：迭代很久不停 / 提示达到 `maxiter` 仍未收敛。- 原因与解决：① 把 `control$maxiter` 调大（如 2000）；② 开 `control$acceleration="squarem"` 加速收敛；③ 若数据信息不足（题太少、人太少、维度太高），模型「弱可识别」，应简化模型或增加题目/样本。

现象：协方差矩阵奇异 / 非正定 / 报错。- 原因：能力分布的协方差估计退化（常见于小样本、某维无题、共线）。- 解决：① 加 `control$ridge = 1e-4`（小岭正则）；② 多组小样本组用 `reg = list(sigma_shrink_pooled = 0.3)` 向总体收缩；③ 检查是否某个维度没有任何题载荷（Q 矩阵某列全 0）。

现象：某题区分度 a 估计成很大的值（跑飞）。- 原因：该题几乎完美区分（「数据分离」），似然在边界，估计不稳。- 解决：用 `reg = list(slope_max = 4)` 设上限，或 `slope_penalty` 加惩罚（这属于流式的「统计正则」，会改变目标函数，软件会标注）。

现象：某道多级题结果异常 / 某类别从没人选。- 原因：GPCM 里某个类别零计数，它的阈值无法识别。- 解决：把这个空类别与相邻类别合并（重新编码），或检查计分是否有误。流式引擎有数值护栏，一般不会崩，但结果该类别不可信。

现象：多维能力对「真值」的相关只有中等（比如 0.5），怀疑算错。- 解释：这通常是数据本身的可识别性问题，不是软件 bug（用标准网格 / 其他软件也是这个结果）。判断软件对不对的标准是「流式引擎 $\approx$ 网格引擎」，而不是「贴近你模拟的真值」。

现象：同样的数据，换了线程数，结果有约 10^{-8} 的微小差异。- 解释：流式引擎把人分块给多个线程并行，再汇总，浮点求和的顺序取决于线程数，导致极微小（可忽略）的差异。要逐位复现就固定 `control$n_threads`；要「跨线程也完全一致」就用 `method="grid"`。这对任何实际结论都无影响。

10.4 8.4 安装与编译类

现象：`install.packages(...)` 时编译报错。- 原因：缺 C++ 编译工具链。- 解决：Windows 装 Rtools；Mac 终端运行 `xcode-select --install`；Linux 装 `r-base-dev`。装好后重新安装 IRTC。

现象：报 `C++17 / CXX_STD` 相关错误。- 解决：升级 R (≥ 3.5) 和编译器；本包需要支持 C++17 的编译器。

现象：`there is no package called 'RcppArmadillo' / 'Rcpp'`。- 解决：先 `install.packages(c("Rcpp", "RcppArmadillo"))` 再装 IRTC。

现象：`there is no package called 'IRTC'` (在 `library(IRTC)` 时)。- 原因：IRTC 没装成功。- 解决：回到第 2.6 节，确认 `install.packages("...tar.gz", repos=NULL, type="source")` 没报错。

现象：`cannot open file ...` / 找不到文件。- 原因：`read.csv` 的路径写错了。- 解决：核对路径 (用 / 不用 \)、文件名、扩展名；或用 RStudio 的 Import Dataset 图形界面。

本章你应该掌握的：遇到任何报错，先复制其中的英文关键词，在本章搜索，按「原因—解决」一步步处理。绝大多数问题源于数据格式 (第 3 章) 或缺工具链 (第 2.6 节)。

11 第 9 章 术语表 (中英对照)

中文	英文	一句话解释
项目反应理论	Item Response Theory (IRT)	用「作答概率」把人的能力和题的性质放在同一把尺子上的理论。
潜在能力 / 特质	latent ability / trait (θ , theta)	看不见、要被测量的东西 (如数学水平、满意度)。
项目 / 题目	item	问卷或测验里的一道题。
作答矩阵	response matrix	「行 = 人、列 = 题」的作答数据表。
二分计分	dichotomous	只有两种结果，编码 0/1。
多级 / 多分计分	polytomous	多个有序等级，编码 0,1,2,...。
难度	difficulty (b) / intercept (xsi)	一道题有多难答对；大 = 难。
区分度	discrimination (a / slope)	一道题分辨能力高低的本事；大 = 好题，负数 = 反向题没反转。
阈值	threshold	多级题里从一档跨到下一档的难度。
维度	dimension	一份问卷测的一个独立特质。
Q 矩阵	Q-matrix	指明「每题属于哪个维度」的 0/1 表。
单维 / 多维	uni- / multidimensional	测一个 / 多个特质。
简单结构 / 题间多维	simple / between-item structure	每道题只属于一个维度。
Rasch / 1PL	Rasch / one-parameter logistic	只估难度、假设各题区分度相同的二分模型。
2PL	two-parameter logistic	估难度和区分度的二分模型。
PCM / GPCM	(generalized) partial credit model	多级题模型，GPCM 还估区分度。
RSM	rating scale model	各题阈值结构相同的多级模型 (适合统一 Likert)。
边际极大似然	marginal maximum likelihood (MML)	把未知能力积分掉后求最大似然的估计方法。
EM 算法	EM algorithm	「猜能力 $\leftrightarrow$ 修题目」来回迭代的求解办法。
似然	likelihood	「某套参数有多像我看到的真数据」。
收敛	convergence	迭代到参数几乎不再变化。
EAP	expected a posteriori	每个人能力的点估计 (科学能力分)。
标准误	standard error (SE)	一个估计有多准；越小越准。
方差 / 协方差	variance / covariance	离散程度 / 两维一起变化的程度。
潜在回归	latent regression	用背景变量解释/预测能力。

中文	英文	一句话解释
多组	multiple groups	不同人群各自一套能力分布。
个案权重	case weights (pweights)	抽样调查里每个人的权重。
信息准则	AIC / BIC	比较模型优劣的分数，越小越好。
偏差	deviance	-2× 对数似然，越小拟合越好。
网格 / 流式引擎	grid / streaming engine	标准全网格 / 大数据省内存的计算引擎。
受控精度	controlled accuracy	用预算换速度、用实测核查保安全的加速。
质量预算	mass budget (ε)	允许剪掉多少先验质量；是预算不是保证误差。
核查报告	accuracy report	加速后实测误差的报告，met=TRUE 表示可信。
R 包	R package	给 R 加功能的插件，IRTC 就是一个。
函数 / 参数	function / argument	一条命令 / 给命令的设置选项。
路径	path / file path	文件在电脑里的「地址」。

12 第 10 章 推荐参考文献与书目

入门教材 (强烈推荐先读) - de Ayala, R. J. (2009). The Theory and Practice of Item Response Theory. Guilford. (公认最友好的 IRT 入门) - Embretson, S. E., & Reise, S. P. (2000). Item Response Theory for Psychologists. Erlbaum. (面向不擅数学的读者)

模型奠基文献 - Rasch, G. (1960). Probabilistic Models for Some Intelligence and Attainment Tests. (Rasch 模型) - Birnbaum, A. (1968). Latent trait models. In Lord & Novick (Eds.), Statistical Theories of Mental Test Scores. (2PL/3PL) - Masters, G. N. (1982). A Rasch model for partial credit scoring. Psychometrika, 47, 149–174. (PCM) - Andrich, D. (1978). A rating formulation for ordered response categories. Psychometrika, 43, 561–573. (RSM) - Muraki, E. (1992). A generalized partial credit model. Applied Psychological Measurement, 16, 159–176. (GPCM)

估计方法 (MML / EM / 多维) - Bock, R. D., & Aitkin, M. (1981). Marginal maximum likelihood estimation of item parameters. Psychometrika, 46, 443–459. (MML/EM 基石) - Adams, R. J., Wilson, M., & Wang, W. (1997). The multidimensional random coefficients multinomial logit model. Applied Psychological Measurement, 21, 1–23. (IRTC 多维框架的来源) - Gibbons, R. D., & Hedeker, D. (1992). Full-information item bifactor analysis. Psychometrika, 57, 423–436. (维度降维思想) - Schilling, S., & Bock, R. D. (2005). High-dimensional MML item factor analysis by adaptive quadrature. Psychometrika, 70, 533–555. - Varadhan, R., & Roland, C. (2008). Accelerating the convergence of EM (SQUAREM). Scandinavian Journal of Statistics, 35, 335–353.

软件 - Chalmers, R. P. (2012). mirt. Journal of Statistical Software, 48(6). - Eddelbuettel, D., & François, R. (2011). Rcpp. Journal of Statistical Software, 40(8).

手册级参考 - van der Linden, W. J. (Ed.) (2016). Handbook of Item Response Theory (3 卷). CRC Press.

13 第 11 章 IRTC 的技术路线、理论基础与优化效果

本章简要说明 IRTC 为什么能处理大型多维数据。一般使用者不必理解底层细节，只需知道：软件在尽量保持统计结果不变的前提下，减少了重复计算和内存占用；只有主动开启 fast=TRUE 时，才使用经过误差核查的近似加速。

13.1 11.1 总体技术路线

IRTC 的优化分为三层：先用 C++ 多线程加速最耗时的 E 步；再利用“每道题只属于一个维度”的简单结构，把题目计算从完整多维网格中拆出来，并按数据块边算边汇总；最后提供可选的节点剪枝，用少量、经过核

查的近似误差换取更大提速。

技术路线	理论基础	实际作用
C++ 多线程与循环重排	并行计算、缓存局部性	多个被试和题目同时计算，减少重复读取和中间复制。
维度分解	题间多维 IRT 的条件独立性	将主要计算量由约 $N \times I \times Q^D$ 降为 $N \times I \times Q + N \times D \times Q^D$ ，题目越多优势越明显。
分块流式 E 步	MML-EM、充分统计量	不保存全部人的完整后验矩阵，内存由随人数增长改为基本受块大小控制。
E 步与统计量融合	EM 算法	后验算完立即汇总题目计数、能力矩和 EAP，避免反复扫描大矩阵。
SQUAREM 与安全化 Newton	固定点加速、梯度/Hessian、线搜索	减少 EM 收敛时间，并提高坏初值或病态数据下的稳定性。
自动选择 grid/streaming	计算复杂度与代价模型	根据人数、题数、维度和内存预测选择更合适的引擎，并在 \$routing 中说明原因。
协变量模式缓存	潜在回归、重复模式复用	相同背景模式的人共用一次先验网格计算，离散协变量几乎不增加额外成本。
受控精度剪枝	数值积分、先验与后验质量	fast=TRUE 时删除不重要节点，并用分层样本实测 EAP、矩、参数和偏差误差。

13.2 11.2 精确性与稳定性原则

IRTC 默认使用精确路径：fast=FALSE 时保留完整积分网格；小数据或不适合流式计算的模型仍可使用 grid。多线程按被试分块，尽量保证不同线程数下结果一致。阻尼、线搜索和正定修复只用于稳定求解，不主动改变统计目标；会改变估计结果的惩罚、参数边界和协方差收缩均需用户显式开启。

13.3 11.3 已达到的效果

在百万样本、多维 GPCM 原型测试中，流式引擎约为 99 秒/次迭代，峰值内存约 2.4 GB；传统完整后验方案在同类规模下可能需要 TB 级内存。流式结果与标准网格结果高度一致，后续版本已支持多组、潜在回归和个案权重，并通过自动化测试与 R CMD check。开启快速模式后，应以 m\$accuracy_report\$met 为准：TRUE 表示分层实测误差达到设定容差，FALSE 时应收紧 mass_budget 或重新使用精确模式。

本章你应该掌握的：① IRTC 的主要提速来自多线程、维度分解和流式计算；② 默认精确模式不使用近似剪枝；③ fast=TRUE 的可信度要看 accuracy_report\$met；④ 大数据优化的核心效果是把原本难以装入内存的多维计算压缩到普通电脑可运行的范围。

14 第 12 章 常见问题解答 (FAQ)

按调查工作者最常问的顺序排列。先扫一眼，多半你的疑问已在其中。

Q1：我完全不会编程，真的能用吗？能。你只需要会「复制本手册的命令、把里面的文件名/数据名换成你自己的、按回车」。第 2、3、6 章手把手教，照做即可。

Q2：我应该用 Rasch(1PL) 还是 2PL/GPCM？- 想要最简单、最稳、不同人群可比、或样本较小 → 用 Rasch/PCM (irtc.mml) - 想让数据自己说话、允许各题区分度不同、判断哪些题区分力强 → 用 2PL/GPCM (irtc.mml.2pl) - 拿不准就两个都跑，用 anova() 或 AIC/BIC 比较 (见 5.6)。

Q3：样本量要多少？经验值：Rasch 类每道题至少 100~200 人较稳；2PL/GPCM 因为多估了区分度，最好 300~500 人以上；多维模型需要更多。样本太小时优先用 Rasch。

Q4：缺失值多了怎么办？少量缺失 (标成 NA) IRT 能自然处理 (用每个人实际作答的题来估计)。但若某人几乎全缺，建议删除该个案；若某题缺失率极高，考虑删除该题。

Q5: Likert 一定要从 0 开始编码吗? 是。多级模型内部按 0,1,2,... 处理。1~5 必须 `dat <- dat - 1` 改成 0~4。这是最常见的出错点。

Q6: 怎么判断结果「跑对了」? - 没有报错、提示收敛; - 区分度没有负数 (有负数 = 反向题没处理); - 难度、阈值在合理范围 (大致 ± 3 内、阈值有序); - 用加速时 `accuracy_report$met == TRUE`。

Q7: EAP 能力分能直接当「分数」用吗? 能。它比原始总分更科学 (考虑了题目难易)。它是类似标准分的量纲 (均值约 0, 正高负低)。如果要给非专业读者看, 可线性变换到熟悉的量纲 (如均值 500、标准差 100): `score <- 500 + 100 * scale(m$person$EAP)`。

Q8: 两个维度该不该合成一个? 看 `cov2cor(m$variance)` 的相关: 相关很高 (如 > 0.9) 可能本就是一个维度; 中等相关 ($0.4 \sim 0.8$) 说明确实是两个相关但可区分的维度, 分开报告更合适。

Q9: 数据很大、跑得很慢怎么办? 用 `method="auto"` (默认会自动选流式引擎), 并开 `control=list(fast=TRUE, refine=TRUE)`。务必看 `accuracy_report$met` 确认误差可接受。

Q10: `method="auto"` 选了 `grid`, 但我觉得 `streaming` 更快? 路由是按预测时间选的, 通常是对的。看 `m$routing$reason`。若你确有把握, 可 `method="streaming"` 强制。一般不必干预。

Q11: 报错了但我看不懂英文? 复制报错里的关键英文词, 去第 8 章搜索。绝大多数报错在那里有「原因 + 解决」。

Q12: 结果每次跑都一样吗? 精确模式 (`fast=FALSE`) 下, 固定数据与设置, 结果逐位可复现。加速模式下若固定 `verify_seed`, 核查报告也可复现; 估计本身的极微小线程级抖动 ($\sim 10^{-8}$) 对结论无影响。

Q13: 我能比较「我这次的题」和「去年那套题」吗? 能, 但需要等值 (`linking/equating`): 两套题要有共同的锚题, 并用 `xsi.fixed` 把锚题参数固定到同一尺度。这属于进阶专题, 建议先读教材 (第 10 章 de Ayala 第 12 章)。

Q14: 什么时候选择 IRTC, 什么时候选择其他 IRT 软件? IRTC 适合需要 MML、流式计算、多组、个案权重或受控精度核查的任务。其他 IRT 软件可能支持不同模型或工作流, 应根据模型范围、数据规模和报告要求选择; 在共同模型上, 正确设定后的结果通常应彼此接近。

Q15: 可以做 3PL (带猜测) 吗? 本包暂不直接支持 3PL。需要时可用 `mirt` 等包。

15 第 13 章 把结果写进报告的模板

直接套用, 把方括号 [...] 换成你的数字。

15.1 14.1 方法部分 (Methods) 模板

「本研究采用项目反应理论 (IRT) 对 [N] 名被试在 [I] 道 [二分/K 级] 题上的作答进行分析。采用 [Rasch/2PL/GPCM] 模型, 经边际极大似然 (MML) 估计 (IRTC 包, [版本])。[多维: 依据理论将题目划分为 [D] 个维度 (见 Q 矩阵)。][反向题第 [...] 题已反向计分。] 模型在 [迭代次数] 次迭代后收敛。」

15.2 14.2 题目质量表 (建议放附表)

题号	难度 b	区分度 a	标准误	判断
1	-0.8	1.2	0.07	良好
...	...	...	...	...
7	0.1	0.21	0.10	区分力低, 建议修订

配一句: 「全部题目区分度为正, 介于 [a_min]-[a_max]; 第 [...] 题区分度低于 0.3, 建议修订或删除。」

15.3 14.3 信度与精度

「量表的边际信度为 [rel], 平均测量标准误为 [mean SD.EAP], 表明 [优秀/可接受/偏低] 的测量精度。测量在中等能力区间最精确, 对极端能力个体精度较低。」

15.4 14.4 人群比较 / 潜在回归

「以 [参照组] 为基准, [另一组] 的平均 [能力] 高/低 $[\Delta]$ 个标准单位。」或: 「潜在回归显示, [协变量] 每增加一个单位, [能力] 平均变化 $[\beta]$ 个标准单位 (控制其他变量)。」

15.5 14.5 加速与可信度 (若用了 fast)

「为处理大样本, 采用受控精度流式估计 (质量预算 $\epsilon=[mass_budget]$)。分层实测核查 ([verify_n] 名分层抽样个体, 含小组、高权重与极端作答者) 确认能力估计的 95% 分位误差为 [eap p95], 远小于估计本身的不确定性, 结果可靠。」

15.6 14.6 一句话写「为什么用 IRT 而不是总分」

「相比简单加总, IRT 将题目难度与被试能力置于同一量尺, 估计不依赖于具体题目集合, 并提供逐人测量精度, 因而更适合本研究的 [比较/等值/精细测量] 目的。」

16 附录

16.1 附录 A: 一页速查卡

```
# ---- 准备 ----
library(IRT)
dat <- as.matrix(read.csv("mydata.csv")) # 读数据
dat <- dat - 1 # Likert 1~5 改 0~4 (按需)
dat[, c(4,9)] <- 4 - dat[, c(4,9)] # 反向题反转 (按需, 0~4)
dat[dat == -9] <- NA # 缺失改 NA (按需)
dim(dat); range(dat, na.rm=TRUE); table(dat) # 体检

# ---- 估计 ----
m1 <- irtc.mml(resp = dat) # Rasch/1PL
m2 <- irtc.mml.2pl(resp = dat, irtmodel = "2PL") # 2PL
m3 <- irtc.mml.2pl(resp = dat, irtmodel = "GPCM") # 多级 GPCM
Q <- cbind(c(1,1,0,0), c(0,0,1,1)) # 多维 Q 矩阵
m4 <- irtc.mml.2pl(resp = dat, irtmodel="2PL", Q = Q) # 多维

# ---- 读结果 ----
summary(m2) # 摘要
m2$xi # 题目难度
m2$person$EAP # 个人能力分
cov2cor(m2$svariance) # 维度相关 (多维)
AIC(m1); AIC(m2) # 比较模型
anova(m1, m2) # 似然比检验

# ---- 大数据加速 ----
m <- irtc.mml.2pl(resp=dat, irtmodel="GPCM", Q=Q,
  control=list(fast=TRUE, mass_budget=1e-3, refine=TRUE))
m$accuracy_report$met # TRUE 才放心
```

16.2 附录 B: CSV 数据模板 (二分题)

把你的数据存成这样的 CSV (第一行是题名, 之后每行一个人):

```
item1,item2,item3,item4
1,0,1,1
1,1,0,1
0,0,1,0
```

多级题同理, 格子放 0,1,2,... (记得从 0 开始)。

16.3 附录 C：新手常用 R 基础操作

```
x <- 5           # 赋值：把 5 存进 x
v <- c(1, 2, 3)  # 造一个向量
m <- matrix(0, 3, 4) # 造一个 3 行 4 列、全 0 的矩阵
dat[1, ]        # 取第 1 行 (第 1 个人)
dat[, 2]        # 取第 2 列 (第 2 道题)
dat[, -(1:2)]   # 去掉前 2 列
nrow(dat); ncol(dat) # 行数、列数
mean(v); sd(v)  # 平均、标准差
?irtc.mml       # 打开某函数的帮助页 (最权威)
```

遇到不会的，永远可以：在控制台输入 `? 函数名` (如 `?irtc.mml.2pl`) 打开官方帮助页；或回到本手册对应章节。

本手册对应 IRTC 0.1.0。函数签名以包内 `?irtc.mml`、`?irtc.mml.2pl` 帮助页为准。如发现手册与软件行为不一致，以软件帮助页为准。